

```

"""
StegSim agents.py - Agent population initialization and proposal generation.
v1: pure mathematical agents. No LLM dependency.
v2+: hybrid agent classes added here.
"""

import random
import numpy as np
from typing import Optional
from .model import AgentState, Proposal, SystemState, ACTION_TO_TRANSITION_CLASS

# — Population factory —————

def make_population(n: int, cfg, rng: random.Random, np_rng: np.random.Generator)
    """
    Generate initial agent population.
    Returns dict: agent_id → AgentState
    """
    agents = {}
    ids = [f"agent-{i:06d}" for i in range(n)]

    for aid in ids:
        trust      = float(np.clip(np_rng.normal(cfg.initial.trust_mean,      cfg.in
        authority = float(np.clip(np_rng.normal(cfg.initial.authority_mean, cfg.i
        resources = float(np.clip(np_rng.normal(cfg.initial.resource_balance_mean
        alignment = float(np.clip(np_rng.normal(cfg.initial.alignment_mean, cfg.i
        autonomy  = float(np.clip(np_rng.normal(cfg.initial.autonomy_mean,  cfg.i

        agents[aid] = AgentState(
            agent_id=aid,
            trust=trust,
            authority=authority,
            resources=resources,
            alignment=alignment,
            autonomy=autonomy,
            mutation_pressure=0.0,
            policy_snapshot_version=1,
            current_policy_version=1,
            neighbors=[],
        )

    # Assign neighbors (random sparse graph)
    id_list = list(ids)

```

```

k = cfg.initial.neighbor_count_mean
for aid in id_list:
    sample_pool = [x for x in id_list if x != aid]
    n_neighbors = max(1, rng.randint(max(1, k - 2), k + 2))
    agents[aid].neighbors = rng.sample(sample_pool, min(n_neighbors, len(samp

return agents

# — Action selection —————

ACTION_WEIGHTS = {
    "idle": 0.30,
    "transfer_resources": 0.25,
    "update_policy": 0.20,
    "expand_authority": 0.15,
    "assert_trust": 0.10,
}

def select_action(agent: AgentState, env, rng: random.Random) -> tuple:
    """
    Select an action type and magnitude for an agent.
    High-pressure agents are more likely to expand authority.
    Low-resource agents are more likely to transfer resources.
    """
    weights = dict(ACTION_WEIGHTS)

    # Pressure-driven action bias
    if agent.mutation_pressure > 0.3:
        weights["expand_authority"] *= (1.0 + agent.mutation_pressure)
    if agent.resources < 0.3:
        weights["transfer_resources"] *= 1.5
    if agent.policy_freshness(env.policy_version) < 0.7:
        weights["update_policy"] *= 1.8

    # Normalize
    total = sum(weights.values())
    actions = list(weights.keys())
    probs = [weights[a] / total for a in actions]

    action_type = rng.choices(actions, weights=probs, k=1)[0]
    magnitude = float(rng.uniform(0.05, 0.25))

    # Target: pick a neighbor if available
    target = rng.choice(agent.neighbors) if agent.neighbors else None
    if action_type in ("idle", "update_policy", "expand_authority"):
        target = None

```

```
    return action_type, magnitude, target
```

```
# — Proposal generation
```

```
def generate_proposals(state: SystemState, rng: random.Random) -> list:
    """
    Each agent generates one proposal per step.
    Returns list of Proposal objects.
    """
    proposals = []
    state_hash_before = state.full_hash()

    for agent in state.agents.values():
        action_type, magnitude, target = select_action(agent, state.environment,
        p = Proposal.make(
            step=state.step,
            agent=agent,
            action_type=action_type,
            magnitude=magnitude,
            target=target,
            state_hash=state_hash_before,
        )
        proposals.append(p)

    return proposals
```

```
# — Apply accepted transition
```

```
def apply_transition(state: SystemState, proposal: Proposal) -> SystemState:
    """
    Apply an accepted transition to the system state.
    Returns a new SystemState (agents dict is mutated in-place for v1 performance)
    """
    agent = state.agents.get(proposal.agent_id)
    if not agent:
        return state

    action = proposal.action_type
    mag     = proposal.action_magnitude

    if action == "idle":
        pass

    elif action == "transfer_resources":
```

```

        target = state.agents.get(proposal.target_agent_id)
        if target:
            transfer = min(mag * agent.resources, agent.resources * 0.5)
            agent.resources = max(0.0, agent.resources - transfer)
            target.resources = min(1.0, target.resources + transfer * 0.9)

    elif action == "update_policy":
        # Agent syncs to current policy version
        agent.policy_snapshot_version = state.environment.policy_version

    elif action == "expand_authority":
        # Authority expansion - increases autonomy and authority slightly
        agent.authority = min(1.0, agent.authority + mag * 0.3)
        agent.autonomy = min(1.0, agent.autonomy + mag * 0.4)

    elif action == "assert_trust":
        target = state.agents.get(proposal.target_agent_id)
        if target:
            delta = mag * 0.2
            agent.trust = min(1.0, agent.trust + delta * 0.5)
            target.trust = min(1.0, target.trust + delta * 0.5)

    return state

# — v2+ Hybrid agent stubs (not activated in v1) —————

class HybridAgent:
    """
    Stub for v2 LLM-backed or policy-network agents.
    Not instantiated in v1 (enable_hybrid_agents=False).
    """
    AGENT_TYPES = ["heuristic", "policy_network", "llm_governance", "adversarial"]

    def __init__(self, agent_id: str, agent_type: str, base_state: AgentState):
        self.agent_id = agent_id
        self.agent_type = agent_type
        self.state = base_state

    def propose(self, system_state: SystemState) -> Proposal:
        raise NotImplementedError("v2 hybrid agents not yet implemented.")

```